IoT for Product Design

From Smart Objects to Ecosystems
From Data to Better Products

Course Guide & Exercise Roadmap

Andrea Vitaletti — Sapienza University of Rome

Course Agreement & Ground Rules

What we agree on from day one:

- **Use of AI is perfectly fine, but it is not an excuse to do not understand**
- **Physical reality matters:** At the end, I want to see a physical product made of real, interconnected things.
- **No validation is unacceptable and dangerous:** If it isn't tested with real humans and real signals, it doesn't work.
- **Validation needs measures:** Provide concrete numbers, latency, error counts, and observations , **not** generic sentences.

1. Course Purpose & Vision

The course introduces product design students to physical computing and IoT using **ESP32** and the open standard **MQTT** (with **ThingsBoard** when needed), driven by two central goals:

1. **From Smart Objects to Ecosystems:**
   Value emerges from the *relationships* between objects, users, and services — not from isolated gadgets.
2. **From Data to Better Products:**
   Sensor data collected in the field informs iterative redesign, maintenance, and user understanding (closing the loop).

- **Target Audience:** Designers with no prior coding experience.
- **Pedagogy:** Step-by-step progression; every technical step ends with a **Design Reflection** and an empirical **Validation**.
- **Going deeper:** technical background in the IoT course notes.

2. Prototyping Strategy: Four Tools

Tool	Role	Why Designers Use It
Wokwi	Browser simulator	Sketching behaviour and wiring with zero risk — **including Wi-Fi & MQTT**
Arduino IDE 2	Desktop environment	Unlimited uploads, physical testing, Serial Monitor/Plotter
MQTTX	MQTT client (desktop / web)	X-ray of the studio: see every message, impersonate other objects
ThingsBoard	IoT platform (demo server)	*Only when necessary:* dashboards for others, history, alarms, settings, OTA

The Golden Rule: Sketch & simulate in **Wokwi** → try on the real board with the **IDE** → use **ThingsBoard** only when the object must serve people who are not its programmers.

📖 Simulating a node · A real node

3. The Running Scenario: "The Connected Studio"

All exercises contribute to a shared, living scenario: bringing a **co-working studio** to life.

- Each student group designs and builds **one studio object**:
  - *Focus Lamp*, *Focus Totem*, *Air Quality Monitor*, *Door / Presence Badge*, *Plant Pot*, *Noise Meter*, *Meeting Gauge*, or *Coffee Counter*.
- **Progressive Evolution:**
  1. **Phase A:** The object acts completely alone (**Smart Object**).
  2. **Phase B:** The object talks MQTT and gets a dashboard (**Connected Object**).
  3. **Phase C:** Objects cooperate directly through shared topics (**Ecosystem**).
  4. **Phase D:** Objects are deployed, gathering real telemetry to justify redesign (**Data-Driven Design**).

3.1 Studio Objects (1/2)

Object	Module(s)	States (Ex 3)	Publishes	Subscribes to
Focus Lamp	LDR or PIR	`OFF` → `FOCUS` → `DO_NOT_DISTURB`	`lamp/mode`	`studioScene`, `co2`
Focus Totem	NeoPixel ring	`AVAILABLE` → `BUSY` → `BREAK_SUGGESTED`	`totem/mode`	`studioScene`, `noiseLevel`
Air Quality Monitor	DHT22, buzzer, *pot* = CO_2	`GOOD` → `STUFFY` → `OPEN_WINDOW`	`co2`	`roomOccupied`
Door / Presence Badge	PIR, buzzer	`EMPTY` → `SOMEONE_ARRIVED` → `OCCUPIED`	`someoneAtDoor`, `roomOccupied`	`studioScene`

Every object uses the **Core Kit** + 1–2 modules, has **≥ 3 states**, and **publishes** at least one topic that **another object subscribes to**. Topics are written without the class prefix `iotpd/studio/`.

3.1 Studio Objects (2/2)

Object	Module(s)	States (Ex 3)	Publishes	Subscribes to
Plant Pot	DHT22 (+ soil probe) or servo	`HAPPY` → `THIRSTY` → `CRITICAL`	`plant/mode`	`studioScene`
Noise Meter	Servo needle, *pot = microphone*	`QUIET` → `LIVELY` → `TOO_LOUD`	`noiseLevel`	`studioScene`
Meeting Gauge	Servo	`FREE` → `IN_USE` → `ENDING_SOON`	`meeting/mode`	`studioScene`
Coffee Counter	NeoPixel ring or buzzer	`IDLE` → `BREWING` → `BREAK_TIME`	`coffee/count`	`roomOccupied`, `totem/mode`

Each object comes with a **design question**, e.g. *Noise Meter:* should a shared object judge people's behaviour? *Door Badge:* where does a greeting end and surveillance begin?

3.2 Simulating CO_2 and Noise with a Potentiometer

The kit has **no CO_2 sensor** (DHT22/BME280 measure only temperature & humidity) and **no microphone** → the Core Kit **potentiometer** plays the environment ("Wizard-of-Oz" sensor).

```
int readCo2()        { return map(analogRead(POT_PIN), 0, 4095, 400, 2000); } // ppm
int readNoiseLevel() { return map(analogRead(POT_PIN), 0, 4095, 0, 100);    } // 0–100
```

- **The sensor is not the design:** the *behaviour* when the room gets stuffy or loud is.
- **Repeatable scenarios:** stage "the room fills up, then someone opens the window" in seconds → essential for **validation**.
- **Works in Wokwi**, which has no CO_2 sensor or microphone.
- **Invisible to the ecosystem:** other objects only see messages on `co2` / `noiseLevel`.
- **Swap later:** a real SCD40 or MAX4466 only changes `readCo2()` / `readNoiseLevel()`.

4. The 2×2 Interaction Foundation

The foundational exercises are built on a symmetrical **2×2 matrix of physical computing primitives** requiring **zero external libraries**:

	Discrete / Binary (0 or 1)	Continuous / Modulated (0 … Max)
Input (Sensing / Control)	**Push Button** (`digitalRead`, `INPUT_PULLUP`)	**Potentiometer** (`analogRead`, 0–4095)
Output (Actuation / Feedback)	**LED On/Off** (`digitalWrite`, HIGH/LOW)	**LED Dimmer** (`analogWrite`, PWM 0–255)

1:1 Bridge to Phone Panels & ThingsBoard Widgets:

- Physical Button ⟷ Panel Button / Switch (publishes a command)
- Physical LED ⟷ LED / Status Indicator
- Physical Potentiometer ⟷ Gauge / Chart
- Physical PWM Dimmer ⟷ Slider (publishes a value)

5. Hardware Kit: Core vs. Project Modules

To prevent beginner frustration (I2C address conflicts, power brownouts, library mismatches), the kit is divided into two tiers:

5.1 Core Kit (Universal — Ex 0 to 2)

- ESP32 DevKit, breadboard, jumper wires, USB cable
- Standard LEDs + current-limiting resistors (220–330 Ω)
- Push buttons (wired straight to GND via internal pull-up)
- Rotary potentiometer (10 kΩ)

5.2 Specialized Project Modules (Introduced in Ex 3)

- **LDR / BH1750:** Ambient light sensing
- **DHT22 / BME280:** Temperature & humidity monitoring
- **PIR Sensor:** Human presence detection
- **RGB NeoPixel Ring:** Chromatic status signaling
- **Servo Motor & Buzzer:** Physical actuation & acoustic alerts

6. Course Structure Overview

Phase	Focus	Core Question	Tools
Phase A (Ex 0–3)	**The Smart Object**	What can an object sense, decide and do alone?	Wokwi, IDE
Phase B (Ex 4–6)	**The Connected Object**	What changes when an object is on the network?	MQTT, MQTTX, ThingsBoard
Phase C (Ex 7–10)	**The Ecosystem**	What emerges when objects cooperate?	Shared MQTT topics
Phase D (Ex 11–14)	**Data-Driven Design**	How can field data improve product and service?	ThingsBoard, Sheets
Final Project (Ex 15)	**Synthesis**	Multi-object connected product-service ecosystem	All

Template for Every Exercise:

Goal → Concepts → Activities → **Design Reflection** → **Validation**

Phase A: The Smart Object

Exercises 0 – 3

Exploring local sensing, actuation, state logic, and the limits of isolation

Ex 0 — Hello Light: GPIO Output & Visual Feedback

- **Goal:** Familiarize with tools and control physical output through an LED.
- **Key Concepts:** Wokwi vs. physical board, Arduino IDE 2, `setup()` / `loop()`, `pinMode()`, `digitalWrite()`, `delay()`.
- **Activities:**
  - Run Blink in Wokwi, change rhythm, add a second alternating LED.
  - Upload sketch to physical ESP32 board.
  - Program distinct product "moods" (calm pulse, alarm, heartbeat).
  - Install the PubSubClient library and MQTTX for Phase B.
- **Design Reflection:** Which blinking patterns are unambiguous to a user without instructions? What differs between simulated and real physical light?
- **Validation:** Test the three mood rhythms with a peer without advance explanation. Can the observer immediately identify which is "calm", "alarm", or "heartbeat"?

📖 Simulating a node · From Wokwi to a real device

Ex 1 — The Tactile Switch: GPIO Input & Interaction

- **Goal:** Read user intention via a push button and build a responsive control loop.
- **Key Concepts:** `digitalRead()`, internal pull-up (`INPUT_PULLUP`), Serial Monitor, conditional logic (`if/else`), momentary vs. latching/toggle state, switch bouncing.
- **Activities:**
  - Wire button to GND (no external resistor required). Inspect pin state.
  - Implement direct momentary feedback (push-to-light).
  - Implement state memory: toggle lamp state ON ↔ OFF per press.
  - Observe physical contact bouncing on hardware; add debouncing.
- **Design Reflection:** Momentary (doorbell) vs. toggle (table lamp) affordances. How critical is instantaneous visual feedback for user confidence?
- **Validation:** Confirm toggle reliability across 20 rapid presses with zero bounce glitches; validate that the physical tactile sensation matches user expectations.

📖 The simplest digital sensor: a button

Ex 2 — The Continuous Dial: Analog Read & PWM

- **Goal:** Capture continuous gestures and produce nuanced modulated output.
- **Key Concepts:** ADC (12-bit, 0–4095), potentiometer voltage divider, `analogRead()`, Serial Plotter, PWM (`analogWrite()`), linear scaling with `map()`.
- **Activities:**
  - Wire potentiometer, read ADC values, inspect smooth curves on Serial Plotter.
  - Dim an LED via PWM duty cycle modulation.
  - Map ADC input (0–4095) to PWM brightness (0–255).
  - Alternative modulation: map knob position to blink tempo (50 ms–1500 ms).
- **Design Reflection:** When is a continuous dial superior to a discrete button? How do friction, throw, and knob resistance communicate product quality?
- **Validation:** Check Serial Plotter for full-range ADC linearity without dead zones; validate with a user if they can dial in a target light level by physical feel alone.

📖 An analog sensor: the potentiometer · Sampling · Filters

Ex 3 — The Isolated Smart Object: State & Non-blocking

- **Goal:** Synthesize primitives into a standalone prototype with distinct modes.
- **Key Concepts:** Sense → Decide → Act loop, Finite-State Machines (`OFF`, `FOCUS`, `ALERT`), non-blocking timing with `millis()`, modular project hardware.
- **Activities:**
  - Eliminate `delay()`: replace with non-blocking timestamp arithmetic.
  - Optional. Integrate chosen Project Module (LDR, PIR, NeoPixel, Servo, or DHT22).
- **Why it matters:** from Ex 4 the object must call `mqtt.loop()` continuously — a long `delay()` makes it deaf to the network.
- **Design Reflection:** Compile the **Isolation Limitations List**: everything the object *cannot* do alone (know room occupancy, alert remotely, report wear).
- **Validation:** Verify zero input lag during continuous LED fading.

📖 More advanced sensors · OS is better

```
for (int b = 0; b <= 255; b++) {    // fade up
  analogWrite(LAMP_PIN, b);
  delay(8);                         // 256 × 8 ms ≈ 2 s of blindness
}
```

```
unsigned long lastToggleAt = 0;
const unsigned long INTERVAL_MS = 500;

void loop() {
  unsigned long now = millis();
  if (now - lastToggleAt >= INTERVAL_MS) {    // has the interval elapsed?
    lastToggleAt = now;
    ledOn = !ledOn;
    digitalWrite(LED_PIN, ledOn);
  }
  // ...everything else (button, pot) runs on every pass, thousands of times per second
}
```

Phase B: The Connected Object

Exercises 4 – 6

Putting the object online with MQTT, remote control, and designing a service

The Mental Model of MQTT

> **"The broker is a post office with labelled mailboxes (topics). An object *publishes* a message to a topic; every object that has *subscribed* to that topic receives a copy. Publishers and subscribers never need to know each other."**

- **Broker:** a public server, e.g. `test.mosquitto.org` — no account, no quota, reachable from Wokwi too.
- **Topic:** a hierarchical name, e.g. `iotpd/studio/lamp/brightness`; wildcards `+` and `#` to listen to many.
- **Payload:** the content — plain text here (`180`, `1`, `focus`), JSON towards ThingsBoard.
- **Retained message:** the broker keeps the last value for whoever subscribes later.
- **Last Will (LWT):** the broker announces `offline` on behalf of an object that disappears.

📖 MQTT · Wireless connectivity

Ex 4 — Going Online: Publish & Watch

- **Goal:** Put the object on the network and let it "speak".
- **Key Concepts:** Wi-Fi, broker, publish/subscribe, topics & payloads, `PubSubClient`, unique client ID, `mqtt.loop()`, MQTTX with `iotpd/studio/#`.
- **Activities:**
  - Subscribe in MQTTX to `iotpd/studio/#`: your "X-ray" of the studio.
  - In Wokwi (`Wokwi-GUEST`), merge Exercise 3 logic into the MQTT template; publish sensor values under `iotpd/studio/<object>/...`.
  - Move the same sketch to the real board (only `secrets.h` changes).
  - Watch the whole class come online on the projector.
- **Design Reflection:** On a public broker your data is visible to *anyone in the world*. What should an object send, and what should it keep to itself?
- **Validation:** Measure latency between a physical change and the message in MQTTX; run a peer privacy audit of what can be deduced by listening to your topics.

📖 Connected by MQTT — Wokwi example

Ex 4 — Code Skeleton

```
#include <WiFi.h>
#include <PubSubClient.h>
#include "secrets.h"                          // WIFI_SSID, WIFI_PASS

const String ROOT = "iotpd/studio/airmonitor/";
WiFiClient net;  PubSubClient mqtt(net);

void setup() {
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  while (WiFi.status() != WL_CONNECTED) delay(500);
  mqtt.setServer("test.mosquitto.org", 1883);
}

void loop() {
  if (!mqtt.connected()) connectMqtt();       // unique client ID per board
  mqtt.loop();                                // never block it with delay()
  if (millis() - lastRead > 10000) {
    lastRead = millis();
    mqtt.publish((ROOT + "temperature").c_str(), String(readTemperature()).c_str());
  }
  updateOutputs();                            // your behaviour from Exercise 3
}
```

Ex 5 — Remote Control: Commands, State & the Phone

- **Goal:** Let the network change the object's behaviour, keeping physical and digital controls consistent.
- **Key Concepts:** Subscribing, the message callback, **command** topics (`lamp/set/brightness`) vs. **state** topics (`lamp/brightness`, retained), phone MQTT panel (e.g. IoT MQTT Panel).
- **Activities:**
  - Subscribe to `<object>/set/#`; test commands by hand from MQTTX.
  - After obeying a command, publish the new state (retained).
  - Build a phone panel: slider → `set/brightness`, gauge ← `brightness`.
  - Pressing the hardware button also publishes the new state.
- **Design Reflection:** Two interfaces, physical & digital: who "wins"? And on a public broker, anyone can publish on your `set/` topics — what should your object refuse?
- **Validation:** Operate knob and phone slider at the same time; validate that both converge without message loops or desynchronization.

📖 How to receive a command on the node

Ex 6 — The Object as a Service: Policies, Resilience & ThingsBoard

- **Goal:** Package the prototype into a reliable, secured, well-designed service.
- **Key Concepts:** Publish policies (*on change* with threshold vs. *periodic*), QoS 0/1, **Last Will** (`<object>/status` online/offline), secrets & TLS, **first ThingsBoard device** (access token, `v1/devices/me/telemetry`) and **dashboard**.
- **Activities:**
  - Tune publish policies to minimize energy and traffic.
  - Add LWT; signal "connecting / offline" on the object itself.
  - Add a second MQTT connection to `demo.thingsboard.io` and send the main values.
  - Design a ThingsBoard dashboard for a specific persona; share it.
- **Design Reflection:** Which part is the product: the object or the service? Why do peers (broker) and people (ThingsBoard) use different channels?
- **Validation:** Persona understands the dashboard in < 30 s; with Wi-Fi off, `status` turns `offline` and the object keeps its local function.

📖 Backend: ThingsBoard · Secure communication

Phase C: The Ecosystem

Exercises 7 – 10

Loose coupling, shared topics, orchestration, and emergent behavior

The Shared Topic Contract

All topics start with the class prefix `iotpd/studio/`. Payloads are plain text.

Topic	Payload	Retained	Purpose	Publisher
`studioScene`	`normal` | `focus` | `presentation` | `closed`	yes	Global room mode	Hub, scene panel
`someoneAtDoor`	`1` (event)	no	Person arrived at the entrance	Door Badge
`roomOccupied`	`0` | `1`	yes	Human presence in room	Presence objects
`co2` / `noiseLevel`	integer	yes	Environmental air/sound quality	Studio monitors
`<obj>/status`	`online` | `offline`	yes	Liveness (Last Will)	Every object
`<obj>/heartbeat`	counter	no	"I am alive" (1/min)	Every object
`<obj>/set/<name>`	depends	no	A **command** to the object	Panels, hub

Design Insight: Designing the topic contract is as fundamental as designing the physical casing!

Ex 7 — Objects Talking to Objects: Shared Topics

- **Goal:** Enable direct multi-device cooperation without centralized code.
- **Key Concepts:** Shared topics, contract adherence, loose coupling (the publisher does not know the subscribers), events vs. commands.
- **Activities:**
  - Pair up with another group (e.g. Door Badge `someoneAtDoor` → Desk Lamp greeting).
  - Subscribe to the partner's topic; **impersonate the partner from MQTTX** until it is ready.
  - Swap partners: connect to a different group's object without editing any firmware.
- **Design Reflection:** What made connecting to another group's object easy or painful? How does a contract mirror physical connector standards?
- **Validation:** Swap partner objects without recompiling; list the ambiguities found in the contract (units, ranges, retained or not, `1` vs `true`).

📖 Devices talking via publish/subscribe

Ex 8 — Who is There? Status, Heartbeats & Resilience

- **Goal:** Make the ecosystem resilient to missing, disconnected, or unpowered devices.
- **Key Concepts:** `status` via Last Will, heartbeat counters, wildcards (`iotpd/studio/+/status`), fallback strategies, **graceful degradation**, ThingsBoard device activity.
- **Activities:**
  - Publish `<object>/heartbeat` every minute and a retained `<object>/capabilities`.
  - Watch partners' `status` and `heartbeat`; switch to local fallback when `offline` or silent for 3 min.
  - Unplug devices at random to test fault tolerance.
  - Build a shared **Studio Map** dashboard in ThingsBoard (image map with live markers).
- **Design Reflection:** Failures are normal. How should an object tell users its partner is missing without appearing "broken"? "Offline" vs. "online but stuck"?
- **Validation:** Unplug partners and verify that local fallback takes over within the timeout; check that the studio map clearly reflects liveness.

Ex 9 — The Orchestrator: Scenes, Hub & Services

- **Goal:** Introduce central coordination and external data feeds.
- **Key Concepts:** Scenes (retained `studioScene`), hub ESP32 subscribing to `iotpd/studio/#` (context rules), time schedules via NTP, web APIs (e.g. Open-Meteo), ThingsBoard alarms & notifications.
- **Activities:**
  - Build a scene panel (e.g. "presentation": dim lamps, silence totems).
  - Hub rule combining inputs (empty room + window open → `windowAlert` + ThingsBoard alarm).
  - Scheduled automation (e.g. evening studio shutdown).
  - External data: hub publishes `weather/rainExpected` → umbrella cue on the entrance lamp.
- **Design Reflection:** Edge vs. hub vs. cloud: which rules belong in the device (fast, private, offline) and which centrally (global view, multi-source context)?
- **Validation:** Trigger multi-device scenes and inspect synchronization; cut the internet uplink and observe which interactions survive.

📖 Public APIs & ThingsBoard weather example

Ex 10 — Ecosystem Design Sprint: Emergence & Conflicts

- **Goal:** Conceive, model, and resolve system-wide ecosystem interactions.
- **Key Concepts:** System mapping, emergent properties, contradictory rule conflicts, user override hierarchies, ThingsBoard **assets & relations** (Studio → Areas → Devices).
- **Activities:**
  - Map all objects, topics, rules and actors; compare with the live topic tree in MQTTX.
  - Model places in ThingsBoard: one asset per area, devices related to their area.
  - Identify and implement an **emergent feature** (e.g. `comfortIndex` computed by the hub).
  - Simulate and resolve a **conflict** (e.g. auto-off rule vs. reading in a corner).
- **Design Reflection:** Compare current capabilities with the limitation list from Exercise 3. What new risks (complexity, privacy, dependency on public servers) were created?
- **Validation:** Stage opposing rules simultaneously to validate override priorities; test whether the comfort index aligns with actual human perception.

📖 Digital twin

Phase D: Data-Driven Design

Exercises 11 – 14

Gathering field telemetry, understanding human use, predictive maintenance, and OTA evolution

Ex 11 — Remembering: Storing & Visualising Data

- **Goal:** Transition from ephemeral live streams to historical design telemetry.
- **Key Concepts:** MQTT does not remember → ThingsBoard **time series**; telemetry (continuous) vs. events (discrete actions), event counters, **instrumentation plans**, CSV/XLS export.
- **Activities:**
  - Draft an instrumentation plan: connect design questions to specific metrics.
  - Instrument the sketch with events and counters (e.g. `dimmerUses`, `event`).
  - Build a historical "Usage Dashboard" with time-window charts.
  - Export CSV regularly (public demo server!); create initial pivot tables.
- **Design Reflection:** Data only answers questions you anticipated. What would you like to know that cannot be sensed? What data must be minimized (GDPR)?
- **Validation:** Compare 24-hour logs against known physical actions to confirm no events were lost; audit metrics against data minimization standards.

📖 Backend: ThingsBoard

Ex 12 — How is it Really Used? Field Deployment

- **Goal:** Deploy in real contexts to extract qualitative & quantitative design insights.
- **Key Concepts:** In-the-wild deployment, usage analytics, identifying "struggle" patterns, comparing intended vs. actual user behavior.
- **Activities:**
  - Deploy objects in the studio, department, or home for **at least 1 week**.
  - Analyze exported logs for non-used features, struggle patterns, or flawed defaults.
  - Conduct short follow-up user interviews to contextualize the numbers.
  - Formulate 3 actionable redesign proposals supported by data.
- **Design Reflection:** Where did data surprise you? Where would data alone have misled you without interviewing real users?
- **Validation:** Cross-validate identified anomalies (e.g. repeated button presses) with qualitative interview quotes to ensure true user intent is understood.

Ex 13 — Keeping it Alive: Health & Maintenance

- **Goal:** Instrument products for diagnostics, reliability, and maintenance.
- **Key Concepts:** Health metrics (uptime, Wi-Fi RSSI, heap memory, battery voltage), wear counters (cycles, on-hours), drift detection, **predictive maintenance**, ThingsBoard **alarm rules** & device inactivity.
- **Activities:**
  - Add wear counters saved across reboots (`Preferences`).
  - Configure alarm rules with severities (critical, major, minor, warning).
  - Build a "Fleet Health" dashboard for the space manager/technician.
- **Design Reflection:** How does maintenance telemetry alter physical industrial design (battery access, antenna placement, modular replacement parts)?
- **Validation:** Simulate hardware faults (voltage drop, signal shielding, frozen sensor); confirm alarms fire at correct severity without notification fatigue.

📖 Maintenance · Energy

Ex 14 — Closing the Loop: Settings & OTA Updates

- **Goal:** Update and improve deployed products remotely based on data insights.
- **Key Concepts:** ThingsBoard **shared attributes** as settings (no new upload), A/B testing in hardware, `firmwareVersion`, Over-The-Air (OTA) firmware deployment (ArduinoOTA or ThingsBoard OTA).
- **Activities:**
  - Expose operational parameters as dashboard sliders (e.g. `autoOffMinutes`, `lightThreshold`).
  - Deploy an A/B test across studio objects to evaluate two feedback variants.
  - Measure variant performance on an A/B analytics dashboard.
  - Deploy the winning firmware over-the-air across the fleet.
- **Design Reflection:** A product is no longer static when sold. What does continuous evolution mean for brand trust, user agency, and ethics?
- **Validation:** Change a setting and confirm instant live adaptation; push an OTA update and verify smooth installation with zero manual intervention.

📖 Over-the-air (OTA) security

Final Synthesis

Exercise 15 & Cross-Cutting Themes

Ex 15 — The Final Capstone Project

- **Brief:** Design a complete **Product-Service Ecosystem** addressing a chosen context (workplace, healthcare, retail, domestic, or cultural spaces):
  1. ≥ 3 **Interconnected Objects:** Sharing a documented topic contract.
  2. **Emergent Value:** Delivering functionality impossible for a single device.
  3. **Instrumentation & Field Study:** Quantitative telemetry + user interviews.
  4. **Data-Driven Iteration:** Demonstrating at least one validated redesign (OTA / settings) justified by data.
- **Validation Requirements:**
  - Live ecosystem demonstration under real environmental conditions.
  - Interoperability test verifying error-free contract compliance.
  - Empirical proof that field data justified the redesign iteration.
  - Critical evaluation of privacy, ethical constraints, and offline degradation.

Cross-Cutting Design Themes

Throughout all phases, keep these strategic questions at the center of the design critique:

- **Privacy & Ethics:** On a public broker *everything* is readable by anyone; data minimization by default.
- **Security:** Wi-Fi credentials, access tokens, TLS, who can publish on your command topics — Security.
- **Sustainability:** Publish policies, sleep modes, longevity, and circular repair.
- **Platform Dependency:** MQTT is an open standard and ThingsBoard is open source: the broker and backend can be swapped. What is the exit plan?
- **Interoperability:** From the studio contract to real standards (Matter, Home Assistant MQTT discovery, Sparkplug).
- **Edge vs. Cloud:** Preserving core product utility when connectivity drops.

Design with Data. Build in Ecosystems.

Ready to prototype with Wokwi, IDE 2, MQTT and ThingsBoard.

Read Full Course Document (intro.md)

Andrea Vitaletti — *IoT for Product Design*